Reviewer Pack — Minimal Rank of Tropicalized Homology Groups vs Resolution P…

Entry ee09026151e0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 06:29:31 UTC

Minimal Rank of Tropicalized Homology Groups vs Resolution Proof Length for Tseitin Formulas

Entry ID: `ee09026151e0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 06:29:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Tropicalization) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

> ['For any given Tseitin formula, the minimal rank of its associated tropicalized homology groups is upper-bounded by the length of a shortest resolution proof for that formula.', 'If F is a Tseitin formula on n variables, then the minimal rank of the tropicalized homology groups of its incidence complex, τ(H(F)), satisfies:', 'τ(H(F)) ≤ O(n^(2/3) log^2(n)) and the resolution proof length for F is Ω(τ(H(F)))']

Rationale (proposer's reasoning):

> ['Tropicalization has been used to simplify algebraic computations and understand combinatorial structures. By studying tropicalized homology groups, we may uncover non-trivial topological properties that correlate with computational complexity in resolution proof systems for Tseitin formulas.', 'The upper bound on the rank is inspired by Gromov-Witten theory in enumerative geometry, which suggests that topological complexities often grow polynomially with geometric parameters.', 'This bridge could potentially reveal new insights into the structure of resolution proofs and provide a novel approach to proving lower bounds for Tseitin formulas.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `44fdf9ac154812e0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Tseitin formulas F on n variables (n ≤ 40), the minimal rank of the tropicalized homology groups τ(H(F)) is less than or equal to O(n^(2/3)

log^2(n)) AND the resolution proof length is greater than or equal to Ω(τ(H(F))). The conjecture is falsified if there exists a Tseitin formula for which τ(H(F)) exceeds O(n^(2/3) log^2(n)) OR the resolution proof length is less than Ω(τ(H(F))).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalization homology groups" AND "resolution proof complexity" - "Tseitin formulas" AND minimal rank tropicalized homology - "incidence complex Tseitin formulas" AND resolution proof length

Top relevant hits considered: - [http://arxiv.org/abs/1710.03219v3] Stabbing Planes - [http://arxiv.org/abs/10 Algebraic Proofs over Noncommutative Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i+1}' for i in range(n)]
        clauses = []
        for var in variables:
            clauses.append([var, f'~{var}'])
        for i in range(1, n):
            clauses.append([f'~{variables[i-1]}', variables[i]])
        return variables, clauses

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for col in range(cols):
            pivot_row = None
            for row in range(col, rows):
                if matrix[row][col] != 0:
                    pivot_row = row
                    break
```

```
            if pivot_row is None:
                continue
            matrix[pivot_row], matrix[col] = matrix[col], matrix[pivot_row]
            for row in range(rows):
                if row == col:
                    continue
                factor = -matrix[row][col] / matrix[col][col]
                for j in range(cols):
                    matrix[row][j] += factor * matrix[col][j]
        rank = sum(1 for row in matrix if any(row[j] != 0 for j in range(cols)))
        return rank

    def resolution_length(clauses):
        stack = clauses[:]
        while True:
            new_clause = None
            for i in range(len(stack)):
                for j in range(i + 1, len(stack)):
                    clause1 = set(stack[i])
                    clause2 = set(stack[j])
                    if any(-x in clause2 for x in clause1):
                        new_clause = [x for x in clause1 ^ clause2 if x > 0]
                        break
                if new_clause is not None:
                    break
            if new_clause is None:
                return len(clauses)
            stack.append(new_clause)

    n = random.randint(5, 40)
    variables, clauses = generate_tseitin_formula(n)
    incidence_matrix = [[1 if var in clause else 0 for var in variables] for clause in clauses]
    rank = gaussian_elimination(incidence_matrix)
    resolution_len = resolution_length(clauses)

    return {
        "metric_name": "Minimal Rank vs Resolution Proof Length",
        "metric_value": rank * resolution_len,
        "instances_tested": 1,
        "conjecture_holds": rank <= n**(2/3) * math.log(n)**2 and resolution_len >= rank,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [random.randint(2, 1000003) for _

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a633339d.py", line 87, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a633339d.py", line 71, in run_trial
    resolution_len = resolution_length(clauses)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a633339d.py", line 58, in resolution_le
    if any(-x in clause2 for x in clause1):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a633339d.py", line 58, in <genexpr>
    if any(-x in clause2 for x in clause1):
           ^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15653
2	preregistration	ollama_remote	glm4:latest	0	0	11072

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	8265
4	novelty	ollama_remote	glm4:latest	0	0	9625
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14253
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10832
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15731
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9930
9	judge	ollama_remote	glm4:latest	0	0	11671

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107031 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ee09026151e0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ee09026151e0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ee09026151e0.tar.gz` (if generated)